Network analysis

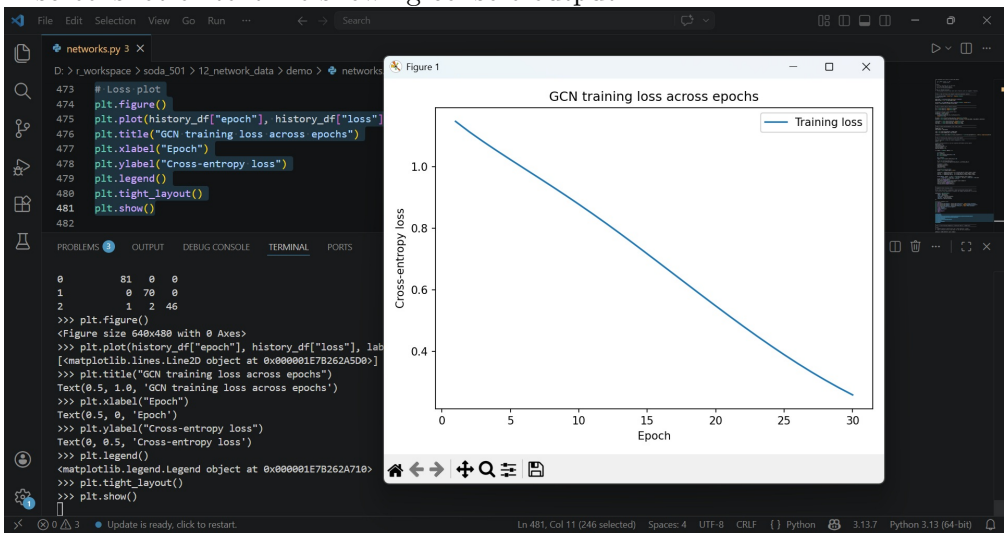
Yasin Shafi

9 April 2026

Note on data. This problem set uses **synthetic** (simulated) network data generated in the provided Python tutorial script. The goal is to practice network workflows (construction, centrality, communities, and GNN-style node classification)—not to draw substantive inferences about real-world actors.

Start Off: Verifying Your Environment

1. **Environment check (required).** Submit proof that you successfully ran the full tutorial code on your machine. You may submit *one* of the following:
   - A screenshot or text file showing console output.



Conceptual Questions

Please write three to ten sentence explanations for each of the following questions. **You are only required to answer ONE of the two questions below.**

2. Networks require representation choices (directed vs. undirected, weighted vs. unweighted, missing edges vs. absent edges, handling isolates). Choose **two** representation choices and explain how each could change a downstream result in this lab (e.g., centrality rankings, community detection, or node classification). Use one concrete example for each.

   Response: Treating the synthetic graph as directed rather than undirected would change betweenness centrality rankings, because a node that frequently receives ties but rarely initi-

ates them could appear as a broker in one direction while being peripheral in the other. For example, if conflict events in ACLED data flow from instigator to target, collapsing directionality would make initiators and recipients look structurally equivalent when they are not. The second choice is how to handle missing edges versus absent edges: if a missing tie is treated as a confirmed zero rather than unknown, the adjacency matrix becomes artificially dense with zeros, which distorts the normalized adjacency used in the GCN and pushes node embeddings toward artificial similarity. In this lab, nodes with many truly unknown connections would receive aggregated neighbor signals as if those neighbors confirmed no relationship, which could cause the GCN to misclassify nodes near community boundaries. Both choices are made before any model runs, which is why there is emphasis that representation decisions shape inference as much as model specification does.

3. Graph neural networks (GNNs) combine node features and network structure via message passing. Explain, conceptually, why using the graph can improve prediction beyond a feature-only model. Then identify one way a GNN can *fail* or mislead in social science settings (e.g., homophily-driven overconfidence, label leakage, missing data, boundary specification).

Applied Exercises

Use the code in the week's code tutorial and the lecture slides to answer the following questions.

Response: Please find this week's work here: `https://github.com/yasinshafi/soda501/tree/main/12_network_data`

4. **Graph construction + centrality.** Using the provided Python script:
   - Generate the synthetic stochastic block model (SBM) network and save (or print) a basic summary: number of nodes, number of edges, and density.
   - Create an edge list (`u`, `v`) and reconstruct the graph from the edge list.
   - Compute three centrality measures:
     (a) degree (or degree centrality),
     (b) approximate betweenness centrality (using sampling),
     (c) eigenvector centrality.
   - Create **one** figure that shows the distribution of each centrality measure (three histograms is fine).
   - Report the top 10 nodes under each measure and briefly discuss (4–8 sentences): do the rankings agree? What kind of "importance" does each metric capture in this synthetic network?

     Response: The three rankings partially overlap but are not identical, which reflects the different concepts of importance each measure captures. Degree and eigenvector centrality agree most strongly: nodes 394, 352, 129, 5, 390, 197, 291, and 390 appear in both top-10 lists, because in an SBM, a node with many connections is also likely connected to other high-degree nodes within the same dense block, making degree and eigenvector reinforce each other. Betweenness ranking looks different - nodes 530, 570, and 252 rank highly there but are absent from the degree and eigenvector top-10s, which suggests they are positionally important as bridges between blocks even though they are not the most locally connected nodes. Eigenvector centrality in this network primarily rewards embeddedness in the largest, densest block (block 0, n=400), since

nodes there are connected to others who are themselves well-connected, amplifying their scores relative to equally well-connected nodes in smaller blocks.

5. **Community detection + evaluation.** Using the same synthetic network:
   - Run Louvain community detection and report:
     (a) the number of detected communities, and
     (b) the sizes of the communities (a small table is fine).
   - Compute the Adjusted Rand Index (ARI) comparing Louvain communities to the known SBM ground-truth labels.
   - Create **one** visualization that communicates the community structure result (e.g., bar plot of community sizes).
   - In 5–8 sentences, interpret what the ARI means here and give one reason why community detection might split or merge true blocks (even when the data are generated from an SBM).

   Response: Louvain detected exactly 3 communities with sizes 400, 350, and 250, which match the true SBM block sizes perfectly. The ARI of 1.0 confirms that every node was assigned to the correct community without a single misclassification. An ARI of 1.0 is the maximum possible value, meaning the Louvain partition and the ground-truth partition are identical up to label permutation. In practice, even with data generated from an SBM, Louvain will not always recover the true blocks this cleanly. One concrete reason is the resolution limit of modularity maximization: when true blocks differ substantially in size, as they do here (400 vs. 250), Louvain can merge the two smaller blocks into one detected community because treating them as a single unit yields a higher modularity score than splitting them, particularly when cross-block tie density is not negligible.

6. **Node classification: features-only baseline vs. GCN-style model.** Using the node features and ground-truth labels in the tutorial script:
   - Fit a features-only baseline model (logistic regression) and report validation and test accuracy.
   - Train the 2-layer GCN-style model from the tutorial and report validation and test accuracy.
   - Create a confusion table (or confusion matrix) for the test set for the GCN model (a table is sufficient).
   - In 6–10 sentences, compare the baseline and GCN results. Your discussion must include:
     (a) why the GCN can outperform the baseline in this setting,
     (b) one reason the baseline might be competitive (or even better) in some settings, and
     (c) one caution about interpreting "high accuracy" as scientific validity in social network prediction tasks.

     Response: The logistic regression baseline substantially outperforms the GCN here, achieving 97.5% test accuracy against the GCN's 63.5% after 30 epochs. The primary reason is that the GCN has not converged: the training loss is still declining at epoch 30 (from 1.107 to 0.738) and training accuracy is only 0.65, which means the model is still learning and the comparison is not between a converged GCN and a converged logistic regression. In principle, the GCN should be able to match or exceed the baseline in this setting because the SBM community structure means a node's neighbors are disproportionately from the same block, so aggregating neighbor features should reinforce the community signal already present in the node's

own feature vector. The confusion table reveals that the GCN struggles most with the boundary between communities 0 and 1, misclassifying 27 true-class-1 nodes as class 0 and 19 true-class-0 nodes as class 1, which are the two largest blocks with the most cross-block edges between them. The baseline can be competitive or better in settings where node features are already highly discriminative relative to the graph topology, and this data is exactly such a setting: the 8-dimensional features were designed with community-specific centers, and with $noise\_sd = 0.8$ the signal-to-noise ratio is high enough that logistic regression can separate the classes cleanly without any graph information. A caution about interpreting the logistic regression's 97.5% accuracy as scientific validity is that both the features and the labels were generated from the same ground-truth community assignments, so the model is essentially recovering structure that was designed to be recoverable rather than discovering something about a real social process. In applied network research, high classification accuracy can reflect circularity in how the network and labels were constructed rather than genuine explanatory insight, and it says nothing about whether the theoretical construct being measured - community membership, influence, conflict - was validly operationalized in the first place.

7. **Challenge Question (Optional — if you finish early):** Run a small sensitivity analysis that varies either **network homophily** or **feature noise**, and compare community detection and prediction performance.
   - Choose **one** knob to vary:
     (a) increase/decrease the off-diagonal probabilities in the SBM matrix P (more vs. less between-community mixing), *or*
     (b) increase/decrease the feature noise standard deviation (`noise_sd`).
   - Run **three** settings (low / medium / high) and record:
     (a) Louvain ARI,
     (b) baseline logistic regression test accuracy,
     (c) GCN test accuracy.
   - Present a small summary table (and optionally a plot) and interpret the pattern (5–10 sentences). In your interpretation, explain when network structure helps most and when it helps least.